GIS Intake Assistant — Step 3 UI / UX

Purpose

Polish the working GIS Intake Assistant into a cleaner drafting workspace without changing the core lookup logic.

This step keeps the working Google Sheets backend and focuses on layout, hierarchy, readability, and day-to-day usability during production.

Step 4 Summary

```
- remove the empty top blocks above each column
- use IBM Plex Mono across the whole page for a more cohesive CAD feel
- expand layout width again so it uses more of the screen
- show saved source notes inside the app, not just in Google Sheets
- restore the built-in copy button for the address block
- fix the broken expander/dropdown arrow icon
- increase spacing and font sizing
- keep Saved Sources always visible
- do not suppress discovery when saved sources exist
- keep Save Verified Source dropdown workflow
- keep Feedback fixed at bottom-right
```

Versioning

Before testing this version:

```
copy app.py versions/app_v0_2_13.py
```

Working version for this step:

```
v0.2.14
```

Full `app.py` — v0.2.14 Step 4 UI / UX Test

```
import pandas as pd
import streamlit as st
import gspread
from google.oauth2.service_account import Credentials
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderTimedOut, GeocoderUnavailable
from ddgs import DDGS
from datetime import datetime


SHEET_NAME = "ED GIS Source Database"
GIS_SOURCES_TAB = "GIS Sources"
SEARCH_LOG_TAB = "Search Log"

SCOPES = [
    "https://www.googleapis.com/auth/spreadsheets",
    "https://www.googleapis.com/auth/drive",
]


# ------------------------------
# Google Sheets connection
# ------------------------------

def get_google_client():
    creds = Credentials.from_service_account_info(
        st.secrets["gcp_service_account"],
        scopes=SCOPES,
    )
    return gspread.authorize(creds)


def get_worksheet(tab_name):
    client = get_google_client()
    sheet = client.open(SHEET_NAME)
    return sheet.worksheet(tab_name)


# ------------------------------
# Load saved GIS / zoning sources
# ------------------------------

def load_gis_portals():
    try:
```

```
        worksheet = get_worksheet(GIS_SOURCES_TAB)
        records = worksheet.get_all_records()
        df = pd.DataFrame(records)

        required_columns = [
            "county", "state", "source_type", "source_name", "source_url",
            "notes", "last_verified", "verified_by", "status",
        ]

        for column in required_columns:
            if column not in df.columns:
                df[column] = ""

        df["county"] = df["county"].astype(str).str.strip().str.lower()
        df["state"] = df["state"].astype(str).str.strip().str.lower()
        df["source_type"] =
df["source_type"].astype(str).str.strip().str.lower()
        df["source_name"] = df["source_name"].astype(str).str.strip()
        df["source_url"] = df["source_url"].astype(str).str.strip()
        df["notes"] = df["notes"].astype(str).str.strip()
        df["status"] = df["status"].astype(str).str.strip().str.lower()

        return df[df["status"].isin(["", "active"])]

    except Exception as error:
        st.error(f"Could not load GIS source database: {error}")
        return pd.DataFrame(columns=[
            "county", "state", "source_type", "source_name", "source_url",
            "notes", "last_verified", "verified_by", "status",
        ])


# ----------------------------
# Save verified source
# ----------------------------

def save_verified_source(county, state, source_type, source_name, source_url,
notes=""):
    df = load_gis_portals()

    county_key = county.strip().lower()
    state_key = state.strip().lower()
    source_type_key = source_type.strip().lower()
    source_url_clean = source_url.strip()

    duplicate = df[
        (df["county"] == county_key)
        & (df["state"] == state_key)
```

```
        & (df["source_url"] == source_url_clean)
    ]

    if not duplicate.empty:
        return False, "This exact source URL is already saved for this county/
state."

    row = [
        county_key,
        state_key,
        source_type_key,
        source_name.strip(),
        source_url_clean,
        notes.strip(),
        datetime.now().date().isoformat(),
        "local_user",
        "active",
    ]

    try:
        worksheet = get_worksheet(GIS_SOURCES_TAB)
        worksheet.append_row(row, value_input_option="USER_ENTERED")
        return True,
"Source saved to Google Sheets. Run lookup again to refresh Saved Sources."
    except Exception as error:
        return False, f"Could not save source: {error}"


# ----------------------------
# Search helpers
# ----------------------------

def is_bad_result(result):
    href = result.get("href", "").lower()
    title = result.get("title", "").lower()

    blocked_terms = [
        "wikipedia", "wikidata", "facebook", "reddit", "linkedin", "youtube",
        "zillow", "realtor.com", "redfin", "homes.com",
    ]

    return any(term in f"{title} {href}" for term in blocked_terms)


def result_matches_place(result, city, county, state):
    title = result.get("title", "").lower()
    href = result.get("href", "").lower()
    body = result.get("body", "").lower()
```

```
    combined_text = f"{title} {href} {body}"

    city_key = city.strip().lower()
    county_key = county.strip().lower()
    county_short = county_key.replace(" county", "")
    state_key = state.strip().lower()

    place_terms = [city_key, county_key, county_short, state_key]
    place_terms = [term for term in place_terms if term]

    return any(term in combined_text for term in place_terms)


def search_candidates(query, city, county, state, max_results=3,
mode="general"):
    results = []

    try:
        with DDGS() as ddgs:
            search_results = ddgs.text(query, max_results=15)

            for result in search_results:
                if is_bad_result(result):
                    continue

                if not result_matches_place(result, city, county, state):
                    continue

                title = result.get("title", "").lower()
                href = result.get("href", "").lower()
                body = result.get("body", "").lower()
                combined_text = f"{title} {href} {body}"

                if mode == "zoning":
                    must_have_zoning = any(term in combined_text for term in [
                        "zoning", "zone district", "zoning map",
                        "zoning viewer", "zoning ordinance", "land use",
                    ])
                    must_be_map_or_official = any(term in combined_text for
term in [
                        "map", "maps", "viewer", "pdf", "arcgis",
                        "municode", ".gov", "planning",
                    ])
                    if not (must_have_zoning and must_be_map_or_official):
                        continue
                else:
                    allowed_keywords = [
                        "gis", "parcel", "assessor", "property",
```

```
                        "map", "maps", "viewer", "arcgis",
                    ]
                    if not any(keyword in combined_text for keyword in
allowed_keywords):
                        continue

                results.append(result)

                if len(results) == max_results:
                    break

    except Exception:
        pass

    return results


def search_general_sources(city, county, state):
    query = f"{county} {state} official GIS parcel viewer"
    return search_candidates(query, city, county, state, max_results=3,
mode="general")


def search_zoning_sources(city, county, state):
    queries = [
        f"{city} {state} official zoning map",
        f"{county} {state} official zoning map PDF",
        f"{city} {state} zoning viewer",
        f"{county} {state} zoning ordinance map",
    ]

    results = []
    seen_urls = set()

    for query in queries:
        candidates = search_candidates(query, city, county, state,
max_results=3, mode="zoning")
        for candidate in candidates:
            href = candidate.get("href", "")
            if href and href not in seen_urls:
                results.append(candidate)
                seen_urls.add(href)

            if len(results) == 3:
                return results

    return results
```

```python
def remove_saved_duplicates(candidates, match):
    if match.empty:
        return candidates

    saved_urls = set(match["source_url"].astype(str).str.strip())
    return [
        candidate for candidate in candidates
        if candidate.get("href", "").strip() not in saved_urls
    ]


# ----------------------------
# Logging
# ----------------------------

def log_search(
    entered_address,
    confirmed_address,
    detected_county,
    detected_state,
    result_type,
    saved_source_count,
    suggested_results_count,
):
    row = [
        datetime.now().isoformat(timespec="seconds"),
        entered_address,
        confirmed_address,
        detected_county,
        detected_state,
        result_type,
        saved_source_count,
        suggested_results_count,
    ]

    try:
        worksheet = get_worksheet(SEARCH_LOG_TAB)
        worksheet.append_row(row, value_input_option="USER_ENTERED")
    except Exception as error:
        st.warning(f"Lookup worked, but search log was not saved: {error}")


# ----------------------------
# Display helpers
# ----------------------------

def display_saved_sources(match):
```

```
    source_labels = {
        "parcel_gis": "Parcel / GIS Map",
        "zoning_map": "Zoning Map",
        "assessor": "Assessor / Property Search",
        "zoning_ordinance": "Zoning Ordinance / Code",
        "other": "Other Source",
    }

    displayed_urls = set()
    displayed_count = 0

    for source_type, label in source_labels.items():
        group = match[match["source_type"] == source_type]

        visible_rows = []
        for _, row in group.iterrows():
            url = str(row["source_url"]).strip()

            if not url or url.lower() == "nan":
                continue

            if url in displayed_urls:
                continue

            visible_rows.append(row)
            displayed_urls.add(url)

        if visible_rows:
            st.markdown(
    """
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=IBM+Plex+Mono:wght@400;500;600;700&display=swap" rel="stylesheet">

    <style>
    body,
    [data-testid="stAppViewContainer"],
    [data-testid="stHeader"],
    [data-testid="stSidebar"],
    h1, h2, h3, h4, h5, h6,
    p, label, a, button, input, textarea,
    div[data-testid="stMarkdownContainer"],
    div[data-testid="stTextInput"] input,
    div[data-testid="stTextArea"] textarea,
    div[data-testid="stForm"] {
        font-family: 'IBM Plex Mono', monospace !important;
    }
```

```
    /* Restore Streamlit / Material icon fonts so copy and expander icons render
properly */
    i.material-icons,
    i.material-symbols-rounded,
    i.material-symbols-outlined,
    span.material-symbols-rounded,
    span.material-symbols-outlined,
    [data-testid="stExpander"] summary span,
    [data-testid="stCodeBlock"] button span {
        font-family: "Material Symbols Rounded", "Material Symbols Outlined",
"Material Icons" !important;
        font-style: normal !important;
        font-weight: normal !important;
        line-height: 1 !important;
        letter-spacing: normal !important;
        text-transform: none !important;
        display: inline-block !important;
        white-space: nowrap !important;
        word-wrap: normal !important;
        direction: ltr !important;
        -webkit-font-smoothing: antialiased !important;
    }

    .block-container {
        padding-top: 2rem;
        padding-bottom: 4rem;
        padding-left: 4rem;
        padding-right: 4rem;
        max-width: none;
    }

    h1 {
        font-size: 2.45rem;
        letter-spacing: -0.04em;
        font-weight: 700;
        margin-bottom: 0.6rem;
    }

    h2 {
        font-size: 1.75rem;
        font-weight: 700;
        letter-spacing: -0.03em;
    }

    h3 {
        font-size: 1.15rem;
        font-weight: 700;
```

```
}

p, .stCaption {
    font-size: 0.95rem;
}

div.stButton > button,
div.stFormSubmitButton > button {
    border: 1px solid #ff9b42;
    background: #11131a;
    color: #ffffff;
    padding: 0.62rem 1.2rem;
    border-radius: 8px;
    font-weight: 600;
}

div.stButton > button:hover,
div.stFormSubmitButton > button:hover {
    border-color: #ffb86b;
    color: #ffb86b;
}

.section-divider {
    border-top: 1px solid rgba(255, 255, 255, 0.16);
    margin: 2rem 0 1.75rem 0;
}

.source-label {
    font-weight: 700;
    margin-top: 1rem;
    margin-bottom: 0.35rem;
    color: #ffffff;
    font-size: 1rem;
}

.source-link,
.candidate-link {
    padding: 0.25rem 0;
    font-size: 0.95rem;
    line-height: 1.55;
}

.source-note {
    opacity: 0.72;
    font-size: 0.82rem;
    margin-left: 1rem;
    margin-bottom: 0.45rem;
    line-height: 1.45;
```

```
    }

    div[data-testid="stExpander"] details {
        border: 1px solid rgba(49, 205, 93, 0.78) !important;
        border-radius: 8px !important;
        background: rgba(255, 255, 255, 0.012);
    }

    div[data-testid="stExpander"] summary {
        font-weight: 600;
    }

    .feedback-button {
        position: fixed;
        right: 2.25rem;
        bottom: 1.75rem;
        z-index: 999;
        border: 1px solid #6ea8ff;
        color: #dce9ff;
        background: #11131a;
        padding: 0.6rem 1.2rem;
        border-radius: 4px;
        text-align: center;
        width: 180px;
        font-size: 0.78rem;
        letter-spacing: 0.08em;
    }
    </style>
    """,
    unsafe_allow_html=True,
)

st.title("GIS Intake Assistant")
st.caption("Find public GIS, parcel, and zoning sources for drafting intake.")

left_col, right_col = st.columns([1, 1], gap="large")

with left_col:
    st.subheader("Project Lookup")

    with st.form("lookup_form"):
        project_address = st.text_input("Project Address",
placeholder="Example: 123 Main St")
        city = st.text_input("City", placeholder="Example: Grand Junction")
        state = st.text_input("State", placeholder="Example: Colorado")
        lookup_submitted = st.form_submit_button("Find GIS Portal")

    if lookup_submitted:
```

```
        full_address = f"{project_address}, {city}, {state}"

        if not project_address or not city or not state:
            st.error("Please enter the project address, city, and state.")
            st.session_state.lookup_result = None
        else:
            geolocator = Nominatim(user_agent="gis_intake_assistant")

            try:
                with st.spinner("Searching public location data..."):
                    location = geolocator.geocode(full_address,
addressdetails=True, timeout=10)

                if location:
                    confirmed_address = location.address
                    address_data = location.raw.get("address", {})
                    detected_county = address_data.get("county", "")
                    detected_state = address_data.get("state", "")

                    county_key = detected_county.strip().lower()
                    state_key = detected_state.strip().lower()

                    gis_df = load_gis_portals()
                    match = gis_df[
                        (gis_df["county"] == county_key)
                        & (gis_df["state"] == state_key)
                    ]

                    with st.spinner("Checking saved sources and discovering
additional sources..."):
                        general_candidates = search_general_sources(city,
detected_county, detected_state)
                        zoning_candidates = search_zoning_sources(city,
detected_county, detected_state)
                        general_candidates =
remove_saved_duplicates(general_candidates, match)
                        zoning_candidates =
remove_saved_duplicates(zoning_candidates, match)

                    saved_count = len(match) if not match.empty else 0
                    suggested_count = len(general_candidates) +
len(zoning_candidates)

                    st.session_state.lookup_result = {
                        "full_address": full_address,
                        "confirmed_address": confirmed_address,
                        "detected_county": detected_county,
                        "detected_state": detected_state,
```

```
                        "match": match,
                        "general_candidates": general_candidates,
                        "zoning_candidates": zoning_candidates,
                        "saved_count": saved_count,
                        "suggested_count": suggested_count,
                    }

                    log_search(
                        entered_address=full_address,
                        confirmed_address=confirmed_address,
                        detected_county=detected_county,
                        detected_state=detected_state,
                        result_type="completed_lookup",
                        saved_source_count=saved_count,
                        suggested_results_count=suggested_count,
                    )

                else:
                    st.warning("No address result found. Try simplifying the
address.")
                    st.session_state.lookup_result = None

            except (GeocoderTimedOut, GeocoderUnavailable):
                st.error("The address lookup service timed out. Try again.")
                st.session_state.lookup_result = None


result = st.session_state.lookup_result

with left_col:
    if result:
        st.success("Location found.")

        st.markdown("<div class='section-divider'></div>",
unsafe_allow_html=True)
        st.subheader("Address to Copy into GIS")
        st.code(result["full_address"], language=None)
        st.caption("Use the copy button on hover to copy/paste the address into
the GIS search bar.")

        st.markdown("<div class='section-divider'></div>",
unsafe_allow_html=True)
        st.subheader("Suggested Additional Sources")
        st.caption("Saved sources appear on the right. These are additional
discovery results to review.")

        st.markdown("**GIS / Parcel Sources**")
        if result["general_candidates"]:
```

```
            display_candidates(result["general_candidates"])
        else:
            query = f"{result['detected_county']}
{result['detected_state']} GIS parcel viewer"
            search_query = query.replace(" ", "+")
            google_search = f"https://www.google.com/search?q={search_query}"
            st.markdown(f"[Fallback GIS Search]({google_search})")

        st.markdown("**Zoning Map Sources**")
        if result["zoning_candidates"]:
            display_candidates(result["zoning_candidates"])
        else:
            query = f"{result['detected_county']} {result['detected_state']}
zoning map PDF zoning viewer"
            search_query = query.replace(" ", "+")
            google_search = f"https://www.google.com/search?q={search_query}"
            st.markdown(f"[Fallback Zoning Map Search]({google_search})")

        render_save_source_form(result["detected_county"],
result["detected_state"])


with right_col:
    st.subheader("Detected Area")

    if result:
        st.write(f"County: {result['detected_county']}")
        st.write(f"State: {result['detected_state']}")
    else:
        st.caption("Run a lookup to detect county and state.")

    st.markdown("<div class='section-divider'></div>", unsafe_allow_html=True)
    st.subheader("Saved Sources")

    if result:
        match = result["match"]
        if not match.empty:
            displayed_count = display_saved_sources(match)

            if displayed_count == 0:

st.warning("Saved source rows found, but no valid unique URLs to display.")
        else:
            st.info("No saved sources found for this county/state yet.")
    else:
        st.caption("Saved sources will appear here after lookup.")
```

```
    st.markdown("<div class='feedback-button'>FEEDBACK</div>",
unsafe_allow_html=True)
```

Run Block

```
streamlit run app.py
```